

MQTT Overview and Benefits with Arduino UNO R4 WiFi

What is MQTT?

MQTT, or Message Queuing Telemetry Transport, is a lightweight messaging protocol for small sensors and mobile devices, to connect through a broker. It was released by IBM in 1999 and is designed for connections with remote devices to provide real-time and reliable messaging services, making it particularly suitable for the Internet of Things (IoT) and small devices.

Benefits of MQTT

- **Low Overhead and Bandwidth Usage:** Ideal for IoT and mobile applications.
- **Supports Continuous Session Control:** Understands that client computing power may be low.
- **Provides Quality of Service Management:** Ensures messages are delivered as needed.
- **Flexible Data Format:** Does not dictate the type or format of the transmitted data.

What Can Arduino UNO R4 WiFi Do with MQTT?

- **Publish Sensor Data:** Send sensor data to designated MQTT topics.
- **Subscribe to Topics:** Receive messages from other devices or services that publish to MQTT topics.
- **Integrate with Smart Home Systems:** Work with MQTT-supported smart home systems for automation control.

Specifically, the Arduino Uno R4 WiFi can communicate with other devices or applications through the MQTT protocol. For example, you can connect it to a device running Home Assistant and create custom home automation devices or sensors through MQTT integration. With MQTT, the Arduino Uno R4 WiFi can publish messages to topics like

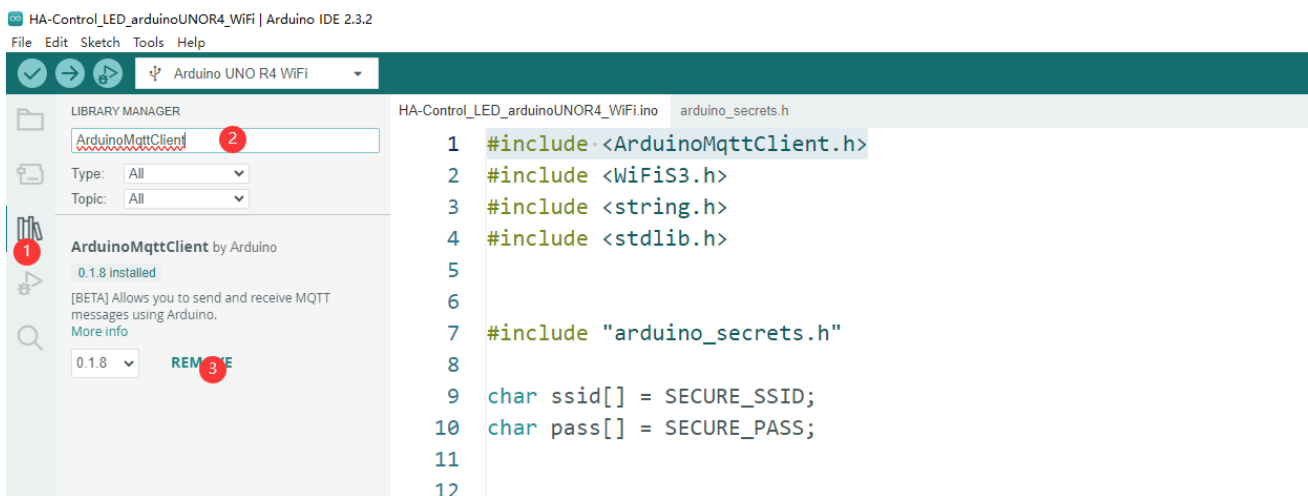
`/sensors/temperature` and also subscribe to messages published by other devices.

Additionally, MQTTX is a full-featured MQTT client tool suitable for desktop, command line, and WebSocket. It helps simplify and make the development and testing of MQTT applications more efficient. In IoT education and development, MQTTX has become a great choice due to its graphical interface and beginner-friendly features.

In summary, MQTT is a communication protocol highly suitable for IoT applications. The Arduino UNO R4 WiFi can easily achieve device-to-device communication and data exchange with MQTT, and the MQTTeX tool provides convenience for the development and testing of MQTT applications.

How to install mqtt library?

- **Open Arduino IDE:** Launch the Arduino Integrated Development Environment (IDE) on your computer.
- **Access Library Manager:** Go to the "Sketch" menu at the top of the IDE, then select "Include Library" and click on "Manage Libraries..." to open the Library Manager.
- **Search for the MQTT Library:** In the Library Manager, there is a search bar at the top. Type "MQTT" into the search bar and press Enter to search for the library.
- **Select the MQTT Library:** From the search results, find the library you want to install. A popular choice is the "ArduinoMqttClient" by Joël Gähwiler.
- **View Library Details:** Click on the library name to view more details about it, including a description, the number of downloads, and user ratings.
- **Install the Library:** On the library details page, click the "Install" button to begin the installation process.



- **Confirm Installation:** A confirmation dialog will appear, click "Confirm" to proceed with the installation.
- **Check Installation:** After the installation is complete, you can check if the library is installed by going to "File" > "Examples" and looking for the MQTT examples in the list.
- **Restart Arduino IDE:** Sometimes, it's necessary to restart the Arduino IDE for the new library to be fully integrated.
- **Start a New Project:** Once the library is installed, you can start a new project by going to "File" > "New" and then adding `#include <ArduinoMqttClient.h>` at the beginning of your sketch to include the MQTT library.
- **Configure MQTT Settings:** In your sketch, you will need to configure the MQTT settings such as the broker address, port, client ID, username, and password if required.
- **Upload Your Sketch:** After configuring the MQTT settings and writing your code, you can upload your sketch to your Arduino board by clicking the "Upload" button.

Demonstration

- Assuming that we now have an MQTT broker server, and also a HomeAssistant server with the address `192.168.3.218`, port `1883`, and the MQTT broker server's access request username is `Jacky`, password is `mypassword`. We want to send the information collected by one of the soil moisture (A2) in our plant watering kit through the MQTT protocol to the MQTT broker server and display the reading information on the Home Assistant page. How should we proceed with such an experiment?

Step 1. Assemble the Hat to Arduino UNO R4 WiFi.

Hardware Overview

The shield provides the following interfaces:

- 3 x Soil Moisture Sensors (Analog Inputs A0, A1, A2)
- 3 x NTC Temperature Sensors (Analog Inputs A4, A5, A6)
- 3 x 3.3V Relay Modules (Digital Outputs 2, 3, 4)
- 3 x Mini Water Pumps
- 1 x 1.3-inch IPS RGB TFT Screen (ST7789 Controller)
- 1 x IR Receiver on board
- 1 x IR remote controller

Pinout Chart

- Details of the expansion board.

Plant Watering Kit Hat Board	Arduino UNO R4 WiFi Board
HUMI3	A0
HUMI2	A1
HUMI1	A2
TEMP3	A3
TEMP2	A4
TEMP1	A5
IR_RSV	D5
Relay 1	D2
Relay 2	D3
Relay 3	D4
TFT_SCLK	D13

Plant Watering Kit Hat Board	Arduino UNO R4 WiFi Board
TFT_MOSI	D11
TFT_CS	D10
TFT_DC	D9
TFT_RST	D8
RX	TX->1
TX	RX<-0
Green LED	D6
Red LED	D7

Connecting the Shield

- place the arduino uno r4 on a flat surface.
- align the shield with the headers of the arduino board and gently press it down until it clicks into place.
- plug the plant watering hat board on top of arduino uno r4 on gpio pins.

Step 2. Create a new sketch

```

sketch_aug14a | Arduino IDE 2.3.2
File Edit Sketch Tools Help
[Icons] Arduino UNO R4 WiFi [Icons]
sketch_aug14a.ino arduino_secrets.h
1 void setup() {
2   // put your setup code here, to run once:
3
4 }
5
6 void loop() {
7   // put your main code here, to run repeatedly:
8
9 }
10
Ln 1, Col 1 Arduino UNO R4 WiFi on COM19

```

Step 3. Programming

- Adding header File

```
#include <ArduinoMqttClient.h>
#include <WiFiS3.h>
#include <string.h>
#include "arduino_secrets.h"
```

- Define variables

```
// define WiFi connection information
char ssid[] = SECURE_SSID;
char pass[] = SECURE_PASS;

// define sensor pin
#define humi1 A2

// setting variables
int status = WL_IDLE_STATUS;
```

- Initialize wifiClient and MqttClient

```
WiFiClient wifiClient;
MqttClient mqttClient(wifiClient);
```

- Define MQTT broker server informations

```
const char broker[] = "192.168.3.218"; // replace this IP to your MQTT broker server's IP address
int port = 1883; // MQTT broker server listen port number.
const char topic1[] = "uno/soil_moisture_ch1"; // mqtt broker's topic name
```

- Modifying setup() function

```

void setup() {
  // Initialize serial port
  Serial.begin(9600);

  // attempt to connect to WiFi network
  Serial.print("attempt to connect to WPA SSID: ");
  Serial.println(ssid);

  while(WiFi.begin(ssid, pass) != WL_CONNECTED) {
    // if failed on connecting wifi, retry again.
    Serial.print(".");
    delay(5000); // wait for 5 seconds and then retry to connect to wifi.
  }

  Serial.println("Connecting to wifi successful");
  Serial.println("More infor:");
  IPAddress ip = WiFi.localIP();
  Serial.print("IP Address: ");
  Serial.println(ip);

  // You can provide a unique client ID, if not set the library uses Arduino-millis()
  // Each client must have a unique client ID
  // mqttClient.setId("clientId");

  mqttClient.setId("Plant_watering_kit");

  // You can provide a username and password for authentication
  // mqttClient.setUsernamePassword("username", "password");

  mqttClient.setUsernamePassword("Jacky", "mypassword");

  Serial.print("Attempting to connect to the MQTT broker: ");

  Serial.println(broker);

  while (!mqttClient.connect(broker, port)) {

    Serial.print("MQTT connection failed! Error code = ");
    Serial.println(mqttClient.connectError());
  }

  Serial.println("You're connected to the MQTT broker!");
  Serial.println();
}

```

- Modifying loop() function

```

void loop() {
  // call poll() regularly to allow the library to send MQTT keep alives which
  // avoids being disconnected by the broker
  mqttClient.poll();

  // reading data from soil moisture
  float raw_data_moisture = analogRead(humi1);

  // print on serial monitor
  Serial.print("Sending message to topic: ");
  Serial.println(topic1);
  Serial.print("Soil moisture raw data:");
  Serial.println(100.0 - (raw_data_moisture / 10.24));

  // send message, the print interface can be used to set the message contents
  mqttClient.beginMessage(topic1);
  mqttClient.print(100.0 - (raw_data_moisture / 10.24));
  mqttClient.endMessage();

  delay(200); // sleep for a while
}

```

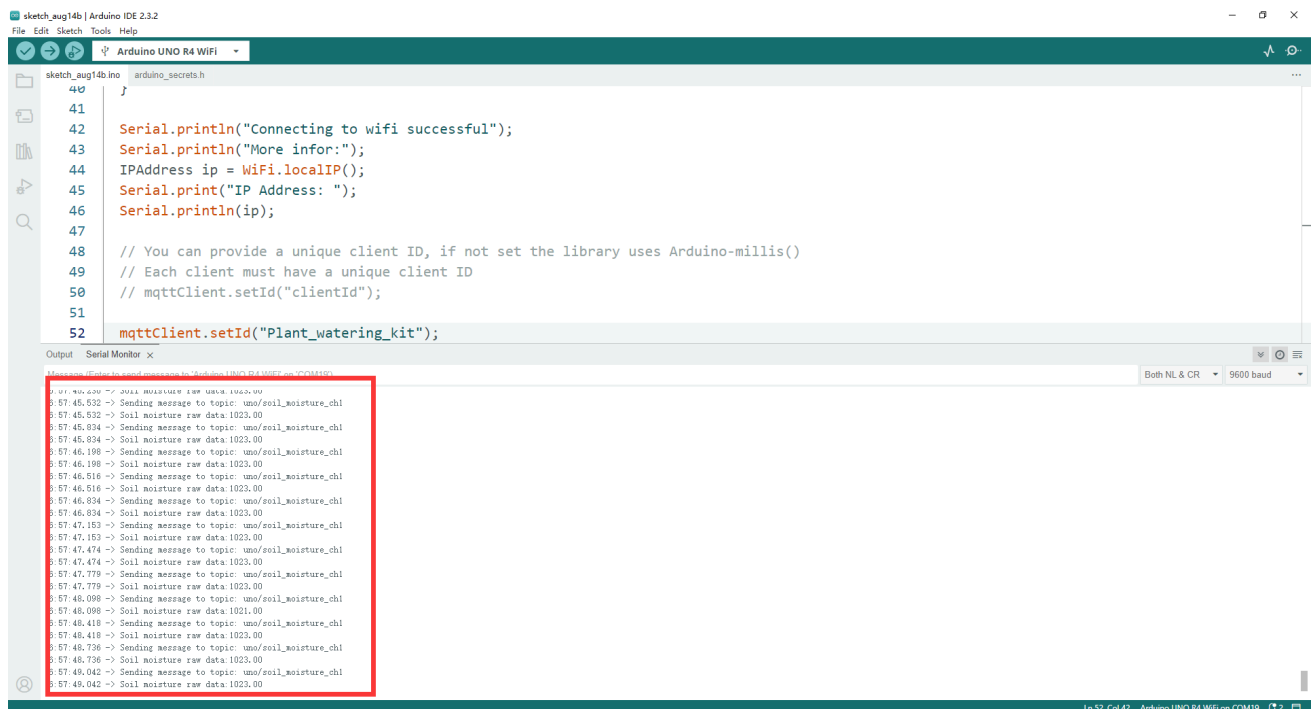
Upload sketch and observe results on Home assistant's dashboard



```
sketch_aug14b.ino
10
11 //define soil moisture sensor pin
12 #define humi1 A2
13
14 // set variable
15 int status = WL_IDLE_STATUS;
16
17 // init wificlient
18 WiFiClient wificlient;
19 MqttClient mqttClient(wificlient);
20
21 // MQTT Broker infor
22 const char broker[] = "192.168.3.218";
23 int port = 1883;
24
```

```
Output
C:\Users\52pidev1\AppData\Local\Arduino15\packages\arduino\tools\arm-none-eabi-gcc\7-2017q4/bin/arm-none-eabi-g++ -c -w -O
Alternatives for WiFi3.h: [WiFi3@0.0.0]
ResolveLibrary(WiFi3.h)
-> candidates: [WiFi3@0.0.0]
C:\Users\52pidev1\AppData\Local\Arduino15\packages\arduino\tools\arm-none-eabi-gcc\7-2017q4/bin/arm-none-eabi-g++ -c -w -O
Compiling sketch...
```

Open Serial monitor



```
sketch_aug14b.ino
40
41
42 Serial.println("Connecting to wifi successful");
43 Serial.println("More infor:");
44 IPAddress ip = WiFi.localIP();
45 Serial.print("IP Address: ");
46 Serial.println(ip);
47
48 // You can provide a unique client ID, if not set the library uses Arduino-millis()
49 // Each client must have a unique client ID
50 // mqttClient.setID("clientId");
51
52 mqttClient.setID("Plant_watering_kit");
```

```
Output Serial Monitor x
11:55:00 AM -> Sending message to topic: umo/soil_moisture_chl
11:57:45.532 -> Sending message to topic: umo/soil_moisture_chl
11:57:45.532 -> Soil moisture raw data: 1023.00
11:57:45.534 -> Sending message to topic: umo/soil_moisture_chl
11:57:45.534 -> Soil moisture raw data: 1023.00
11:57:46.198 -> Sending message to topic: umo/soil_moisture_chl
11:57:46.198 -> Soil moisture raw data: 1023.00
11:57:46.516 -> Sending message to topic: umo/soil_moisture_chl
11:57:46.516 -> Soil moisture raw data: 1023.00
11:57:46.534 -> Sending message to topic: umo/soil_moisture_chl
11:57:46.534 -> Soil moisture raw data: 1023.00
11:57:47.153 -> Sending message to topic: umo/soil_moisture_chl
11:57:47.153 -> Soil moisture raw data: 1023.00
11:57:47.474 -> Sending message to topic: umo/soil_moisture_chl
11:57:47.474 -> Soil moisture raw data: 1023.00
11:57:47.779 -> Sending message to topic: umo/soil_moisture_chl
11:57:47.779 -> Soil moisture raw data: 1023.00
11:57:48.098 -> Sending message to topic: umo/soil_moisture_chl
11:57:48.098 -> Soil moisture raw data: 1021.00
11:57:48.418 -> Sending message to topic: umo/soil_moisture_chl
11:57:48.418 -> Soil moisture raw data: 1023.00
11:57:48.736 -> Sending message to topic: umo/soil_moisture_chl
11:57:48.736 -> Soil moisture raw data: 1023.00
11:57:49.042 -> Sending message to topic: umo/soil_moisture_chl
11:57:49.042 -> Soil moisture raw data: 1023.00
```

HomeAssistant server web admin page

The screenshot displays the Home Assistant web interface at the URL `192.168.3.218:8123/lovelace/default_view`. The interface includes a sidebar with navigation options: Overview, Energy, Map, Logbook, History, HAACS, Media, and To-do lists. The main content area shows a 'Home' dashboard with three cards: a user card for 'jacky.li' (Unknown), a weather card for 'Partly cloudy' (Forecast Home) with a temperature of 31.1 °C and 64% humidity, and a soil moisture card for 'Soil Moisture CH1' showing 32.81% with a red arrow pointing to the right.

Overlaid on the interface is an Arduino IDE window titled 'sketch_aug14b | Arduino IDE 2.3.2'. The window shows the 'Serial Monitor' tab with the following output:

```
17:22:12.688 -> Soil moisture raw data:28.61
17:22:13.006 -> Sending message to topic: uno/soil_moisture_ch1
17:22:13.006 -> Soil moisture raw data:29.49
17:22:13.328 -> Sending message to topic: uno/soil_moisture_ch1
17:22:13.328 -> Soil moisture raw data:29.88
17:22:13.648 -> Sending message to topic: uno/soil_moisture_ch1
17:22:13.648 -> Soil moisture raw data:30.57
17:22:13.918 -> Sending message to topic: uno/soil_moisture_ch1
17:22:13.918 -> Soil moisture raw data:30.96
17:22:14.271 -> Sending message to topic: uno/soil_moisture_ch1
17:22:14.271 -> Soil moisture raw data:31.15
17:22:14.589 -> Sending message to topic: uno/soil_moisture_ch1
17:22:14.589 -> Soil moisture raw data:31.45
17:22:14.907 -> Sending message to topic: uno/soil_moisture_ch1
17:22:14.907 -> Soil moisture raw data:31.64
17:22:15.225 -> Sending message to topic: uno/soil_moisture_ch1
17:22:15.225 -> Soil moisture raw data:32.42
17:22:15.543 -> Sending message to topic: uno/soil_moisture_ch1
17:22:15.543 -> Soil moisture raw data:32.81
```

The last three lines of the serial monitor output are highlighted with a red box. The status bar at the bottom of the IDE window indicates 'Ln 85, Col 53' and 'Arduino UNO R4 WiFi on COM19'.

Full demo code

```
#include <ArduinoMqttClient.h>
#include <WiFiS3.h>
#include <string.h>
#include "arduino_secrets.h"

//please enter your sensitive data in the Secret tab/arduino_secrets.h
char ssid[] = SECRET_SSID; // your network SSID (name)
char pass[] = SECRET_PASS; // your network password (use for WPA, or use as key for WEP)

//define soil moisture sensor pin
#define humi1 A2

// set variable
int status = WL_IDLE_STATUS;

// init wificlient
WiFiClient wifiClient;
MqttClient mqttClient(wifiClient);

// MQTT Broker infor
const char broker[] = "192.168.3.218";
int port = 1883;

// topic infor
const char topic1[] = "uno/soil_moisture_ch1";

void setup() {
  // Initialize serial port
  Serial.begin(9600);

  // attempt to connect to WiFi network
  Serial.print("attempt to connect to WPA SSID: ");
  Serial.println(ssid);

  while (WiFi.begin(ssid, pass) != WL_CONNECTED) {
    // if failed on connecting wifi, retry again.
    Serial.print(".");
    delay(5000); // wait for 5 seconds and then retry to connect to wifi.
  }

  Serial.println("Connecting to wifi successful");
  Serial.println("More infor:");
  IPAddress ip = WiFi.localIP();
  Serial.print("IP Address: ");
  Serial.println(ip);

  // You can provide a unique client ID, if not set the library uses Arduino-millis()
  // Each client must have a unique client ID
  // mqttClient.setId("clientId");

  mqttClient.setId("Plant_watering_kit");

  // You can provide a username and password for authentication
  // mqttClient.setUsernamePassword("username", "password");

  mqttClient.setUsernamePassword("jacky", "mypassword");

  Serial.print("Attempting to connect to the MQTT broker: ");

  Serial.println(broker);

  while (!mqttClient.connect(broker, port)) {
    Serial.print("MQTT connection failed! Error code = ");
    Serial.println(mqttClient.connectError());
  }

  Serial.println("You're connected to the MQTT broker!");
  Serial.println();
}

void loop() {
```

```
// call poll() regularly to allow the library to send MQTT keep alives which
// avoids being disconnected by the broker
mqttClient.poll();

// reading data from soil moisture
float raw_data_moisture = analogRead(hum11);

// print on serial monitor
Serial.print("Sending message to topic: ");
Serial.println(topic1);
Serial.print("Soil moisture raw data:");
Serial.println(100.0 - (raw_data_moisture / 10.24));

// send message, the print interface can be used to set the message contents
mqttClient.beginMessage(topic1);
mqttClient.print(100.0 - (raw_data_moisture / 10.24));
mqttClient.endMessage();

delay(200); // sleep for a while
}
```

Demo code download

- Download: [Basic_8_MQTT_basic](#)

Demo Video

